Residual Dependency Management Module (RDMM)

Residual Dependency Management Module (RDMM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Retirement Standard (SRTS)

Category: Governance & Enforcement

Subcategory: Software Retirement Governance

Type: Core Module

Governed Space: Software Retirement Residual Dependency Governance

Module ID: SLGA-SRTS-RDMM-04

Origin ID: OOF-SLGA-SRTS-RDMM

Version: 1.0

Status: Canonical Open

Effective Date: 15.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: UCL — Universal Canonical Language (English)

Canonical Definition

Residual Dependency Management Module (RDMM) is the Core Module within Software Retirement Standard (SRTS) that governs how dependencies, operational relationships, technical connections, data requirements, authority conditions, and continuing obligations that remain during or after software withdrawal are identified, assessed, classified, assigned, controlled, resolved, transferred, or explicitly retained under governance.

RDMM establishes residual dependency management as an evidence-supported

governance process addressing the continuing relationships between a retiring software object and other software, systems, users, infrastructure, data, operational processes, or external actors.

The module distinguishes dependency identification from dependency

verification, software withdrawal from dependency elimination, dependency transfer from dependency resolution, and the absence of observed activity from verified dependency termination.

It requires residual dependencies to remain connected to identifiable

software objects, their originating relationships, affected operational environments, continuing functions, responsible actors, applicable authority, resolution requirements, and supporting evidence.

RDMM recognizes that software may be deactivated, replaced, unsupported, or administratively retired while important dependencies continue to exist.

These dependencies may include active integrations, historical data access, shared infrastructure, recovery mechanisms, scheduled processes, retained credentials, autonomous agent permissions, external consumers, embedded software relationships, or unresolved operational responsibilities.

RDMM also recognizes that not every residual dependency must be eliminated. Some may require legitimate retention, controlled restriction, reassignment, replacement, or continued monitoring.

The module governs the disposition of these relationships without assuming

that their existence automatically prevents every form of retirement closure.

It establishes the conditions under which a residual dependency can be considered resolved, transferred, restricted, retained, or unresolved.

Its principal governance output is the Software Residual Dependency Profile.

Operational Role

RDMM establishes the governance framework for dependencies that remain relevant during or after software retirement activities.

It governs:

• Residual dependency case identity

• Residual dependency discovery

• Dependency origin

• Source software identification

• Dependent object identification

• Dependency relationship classification

• Active and inactive dependency conditions

• Direct and indirect dependencies

• Technical dependencies

• Operational dependencies

• Data dependencies

• Interface dependencies

• Infrastructure dependencies

• Security dependencies

• Authority dependencies

• Autonomous system dependencies

• Historical and recovery dependencies

• Dependency significance

• Dependency impact

• Dependency ownership

• Responsibility assignment

• Dependency verification

• Dependency resolution requirements

• Dependency replacement

• Dependency transfer

• Dependency restriction

• Legitimate dependency retention

• Dependency monitoring

• Residual risk

• Dependency exceptions

• Dependency reassessment

• Resolution evidence

• Residual dependency closure

• Handoff to retirement closure governance

RDMM does not independently establish final software retirement closure.

Operational Question

How can dependencies and continuing operational relationships remain identifiable, controlled, and accountable after software is withdrawn, replaced, or declared retired, particularly when some relationships must be eliminated while others must legitimately continue?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SRTS Core Module Flow

Retirement Planning Module (RPM) → End-of-Support Management Module (ESMM)

→ Software Migration Module (SMM) → Residual Dependency Management Module (RDMM) → Retirement Closure Control Module (RCCLM)

RDMM is the fourth Core Module within SRTS.

It receives dependency conditions identified during retirement planning,

support transition, migration, and software withdrawal.

It governs the remaining dependency relationships and establishes their

appropriate disposition.

Its outputs provide essential evidence for determining whether software

retirement can legitimately proceed to closure.

• Governed Space

Software Retirement Residual Dependency Governance

This governed space concerns dependencies and continuing relationships

associated with software undergoing or having undergone retirement activities.

It begins when a dependency is identified as potentially remaining relevant beyond a planned or executed software transition.

It includes dependency discovery, identification, classification,

significance assessment, responsibility, resolution requirements, authority conditions, verification, and continuing governance.

It remains applicable where dependencies are unresolved, conditionally controlled, retained for legitimate purposes, or subject to ongoing obligations.

It concludes for an individual dependency when its disposition has been established through appropriate evidence and any continuing responsibilities have been assigned or transferred.

Governed Object

The primary Governed Object is:

A Software Residual Dependency Case representing the evidence-supported

relationship between an identifiable retiring software object, a dependent or supporting object, the dependency's operational purpose, its actual continuing condition, associated risks and obligations, applicable authority, responsible actors, intended disposition, verification evidence, and resulting governance state.

The governed object must preserve, where applicable:

Residual Dependency Case ID

Retirement Case Reference

Source Software ID

Dependent Object ID

Dependency Origin

Dependency Type

Dependency Direction

Dependency Purpose

Current Dependency State

Operational Significance

Affected Environment

Continuing Obligation

Responsible Function

Applicable Authority

Required Disposition

Resolution Conditions

Verification Evidence

Residual Risk

Dependency Status

Record Version

Scope

RDMM governs:

Residual Dependency Discovery

Residual Dependency Identification

Dependency Relationship Mapping

Dependency State Assessment

Dependency Significance Assessment

Operational Impact Assessment

Dependency Ownership Identification

Dependency Responsibility Assignment

Dependency Resolution Planning

Dependency Elimination

Dependency Replacement

Dependency Transfer

Dependency Restriction

Dependency Retention

Dependency Verification

Dependency Monitoring

Residual Risk Identification

Dependency Exception Governance

Dependency Reassessment

Dependency Closure

Retirement Closure Handoff

RDMM does not replace general software dependency governance.

It governs the residual dependency conditions specifically arising from or

remaining relevant to software retirement.

Foundational Principle

Software retirement cannot be evaluated solely through the state of the retiring software object; it must also account for the operational relationships that continue to depend on, reference, access, invoke, preserve, or remain affected by that object.

A software artifact may be removed while its operational consequences remain.

A dependency may be inactive without being eliminated.

A dependency may be transferred without being verified.

And a dependency may legitimately remain without invalidating retirement, provided its continuing condition is explicitly governed.

Software Removed ≠ Dependency Eliminated.

Dependency Retained ≠ Dependency Ungoverned.

Residual Dependency Case Identity

Every material residual dependency must have a stable case identity.

Relevant elements include:

Dependency Case ID

Retirement Case Reference

Source Software Reference

Dependent Object Reference

Dependency Type

Discovery Origin

Responsible Function

Creation Time

Dependency Status

Evidence Reference

Record Version

The identity must remain stable across discovery, assessment, resolution, transfer, and closure.

Residual Dependency Origin

Residual dependencies may originate from:

Software Architecture

Historical Integration

Runtime Configuration

Shared Infrastructure

Data Relationships

External Consumers

Scheduled Automation

Recovery Arrangements

Security Permissions

Operational Workflows

Physical-System Integration

Migration Incompleteness

Support Transition

Software Withdrawal

The originating relationship must remain traceable where evidence is available.

Dependency Object Identification

RDMM must identify the objects participating in the residual dependency.

Potential objects include:

Software Application

Software Component

Library

Runtime Platform

Database

API

External Service

Infrastructure Component

Automated Workflow

AI Agent

User Account

Service Identity

Embedded Device

Recovery System

Historical Archive

The relationship must be identified with sufficient precision to support governance action.

Dependency Direction

RDMM must establish the direction of dependency.

Potential relationships include:

Retiring Software Depends on External Object

External Object Depends on Retiring Software

Bidirectional Dependency

Shared Dependency

Indirect Dependency

Historical Dependency

Conditional Dependency

Dependency direction must not be inferred solely from ownership or deployment location.

Dependency Relationship

Every material dependency must preserve its operational relationship.

Relevant elements include:

Source Object

Dependent Object

Required Capability

Interaction Mechanism

Dependency Direction

Operational Environment

Execution Conditions

Failure Consequences

Evidence Reference

A listed connection must not automatically be treated as a verified operational dependency.

Dependency Purpose

RDMM must identify why the dependency exists or existed.

Potential purposes include:

Functional Execution

Data Access

Authentication

Authorization

Communication

Configuration

Monitoring

Logging

Recovery

Historical Reporting

Physical Control

Automated Decision Support

Regulatory Record Access

Understanding dependency purpose is necessary to determine whether elimination, replacement, transfer, or retention is appropriate.

Dependency Classification

Residual dependencies may be classified as:

Technical Dependency

Operational Dependency

Functional Dependency

Data Dependency

Interface Dependency

Infrastructure Dependency

Security Dependency

Authority Dependency

Identity Dependency

Recovery Dependency

Historical Dependency

External Dependency

Physical-System Dependency

Autonomous Execution Dependency

A single dependency may belong to multiple categories.

Direct Residual Dependencies

Direct residual dependencies connect identifiable objects without requiring

an intermediate dependency relationship.

Potential examples include:

Application to Database

Service to API

Agent to Tool

Device to Firmware

User Workflow to Legacy Application

Backup Process to Source System

Direct dependencies must remain associated with their actual interaction mechanism.

Indirect Residual Dependencies

Indirect residual dependencies exist through intermediate systems or relationships.

A retiring software service may support another service that, in turn, supports a critical operational process.

RDMM must identify:

Intermediate Objects

Dependency Path

Affected Functions

Operational Consequences

Known Uncertainty

Verification Evidence

Indirect dependencies must not be ignored merely because they do not interact directly with the retiring software.

Transitive Dependency Relationships

RDMM must recognize transitive relationships where a dependency propagates across multiple software or infrastructure objects.

Relevant elements include:

Dependency Chain

Intermediate Software

Shared Resources

Critical Path

Failure Propagation

Affected Operational Function

Resolution Requirements

The significance of a residual dependency may arise from its position within a broader dependency chain.

Runtime Dependencies

Runtime dependencies are resources required for software execution.

Potential objects include:

Runtime Libraries

Execution Environments

Container Services

Operating Systems

Runtime APIs

External Services

Configuration Services

Authentication Services

RDMM must determine whether these relationships remain relevant after software withdrawal.

Infrastructure Dependencies

Infrastructure dependencies may include:

Servers

Cloud Resources

Storage Systems

Networks

Load Balancers

Message Brokers

Monitoring Systems

Identity Services

Backup Infrastructure

A shared infrastructure component must not be removed solely because one dependent software object has been retired.

Data Dependencies

Data dependencies may remain after software execution ends.

Relevant elements include:

Historical Data

Operational Records

Database Relationships

Archived Data

Reporting Data

Audit Evidence

Recovery Data

Data Transformation References

Data Access Tools

RDMM must identify whether continued data accessibility depends on the retiring software.

Historical Data Access Dependencies

Some retired software may remain necessary to interpret or access historical records.

Relevant elements include:

Historical Data Object

Required Software Capability

Data Format

Access Mechanism

Retention Requirement

Responsible Function

Alternative Access Method

Verification Evidence

Historical data retention does not automatically require continued operation of the original application.

Interface Dependencies

Residual interface dependencies may include:

API Consumers

Legacy Endpoints

Webhooks

Message Queues

File Exchange Interfaces

Authentication Integrations

Industrial Protocols

External Partner Connections

RDMM must identify whether consumers continue attempting to use interfaces associated with retiring software.

External Consumer Dependencies

External consumers may remain dependent on software after internal retirement activities.

Relevant elements include:

Consumer Identity

Consumed Capability

Connection Method

Operational Frequency

Known Obligations

Communication Requirements

Replacement Interface

Verification Evidence

Unknown consumers must remain explicit where discovery is incomplete.

Operational Process Dependencies

Software may remain connected to business or operational processes.

Potential relationships include:

Financial Reporting

Production Planning

Inventory Control

Customer Service

Compliance Reporting

Maintenance Scheduling

Operational Monitoring

Emergency Recovery

RDMM must assess whether the process remains dependent on the original software capability.

Scheduled Execution Dependencies

Residual scheduled activities may remain active after software retirement.

Relevant elements include:

Scheduled Task Identity

Execution Trigger

Responsible System

Referenced Software

Execution Authority

Expected Output

Actual Execution State

Termination Evidence

Scheduled tasks must not be considered terminated merely because the main application has been deactivated.

Automation Dependencies

Automated systems may continue referencing retiring software.

Potential relationships include:

CI/CD Pipelines

Infrastructure Automation

Monitoring Automation

Scheduled Maintenance

Data Synchronization

Backup Automation

Security Scanning

External Orchestration

RDMM must identify the actual execution paths and dependent actions.

Autonomous Agent Dependencies

Autonomous agents may retain dependencies on software that has been replaced or retired.

Relevant elements include:

Agent Identity

Assigned Objectives

Tool Dependencies

External APIs

Memory References

Scheduled Actions

Delegated Authority

Persistent Credentials

Operational Responsibilities

Agent dependency governance must preserve the distinction between technical access and legitimate authority.

Authority Dependencies

Authority dependencies arise where operational activities rely on permissions, delegations, or access relationships associated with retiring software.

Relevant elements include:

Identity Reference

Permission

Delegation

Authorized Objective

Tool Access

External System Access

Validity Condition

Revocation Requirement

Successor Authority

An unused permission may remain a material residual dependency.

Identity Dependencies

Residual identity relationships may include:

Service Accounts

Application Identities

Agent Identities

Certificates

API Credentials

Authentication Tokens

Trust Relationships

External Identity References

RDMM must determine whether these relationships remain active, necessary, transferable, or subject to termination.

Security Dependencies

Security-related dependencies may include:

Authentication Services

Authorization Policies

Security Monitoring

Encryption Keys

Certificate Chains

Vulnerability Scanning

Incident Response Integration

Access Revocation Mechanisms

The retirement of software must not unintentionally eliminate security capabilities required by other systems.

Recovery Dependencies

Recovery dependencies may remain necessary after software withdrawal.

Potential objects include:

Backup Systems

Recovery Images

Legacy Restore Tools

Archived Configurations

Disaster Recovery Workflows

Historical Deployment Artifacts

Data Recovery Interfaces

RDMM must identify whether these dependencies require continued retention or replacement.

Physical-System Dependencies

Where software interacts with physical equipment, residual dependencies may include:

Embedded Controllers

Firmware

Industrial Devices

Robotic Systems

Safety Mechanisms

Monitoring Equipment

Calibration Tools

Maintenance Interfaces

The absence of active software execution does not automatically establish that physical-system dependencies have been eliminated.

Residual Dependency Discovery

RDMM must establish an appropriate discovery process.

Potential discovery sources include:

Software Inventories

Dependency Maps

Runtime Monitoring

Network Records

Configuration Records

Source Code References

Deployment Records

Access Logs

Scheduled Task Registers

Data Flow Records

Operational Interviews

External Consumer Records

Discovery methods must be proportionate to the operational significance of the retirement case.

Discovery Completeness

RDMM must evaluate whether dependency discovery is sufficiently complete for the intended governance decision.

Relevant considerations include:

Known Environments Covered

Relevant Software Versions Covered

External Consumers Assessed

Scheduled Activities Assessed

Data Relationships Assessed

Authority Relationships Assessed

Recovery Dependencies Assessed

Uncertainty Recorded

Discovery completeness must not be assumed merely because no additional dependencies were found.

Dependency Evidence

Material dependency conclusions must be supported by appropriate evidence.

Potential evidence includes:

Runtime Observations

Configuration References

Source Code Analysis

Network Activity

Access Records

Dependency Manifests

Operational Documentation

Migration Records

System Owner Confirmation

Execution Logs

Evidence must remain associated with the dependency state and period to which it applies.

Dependency State Assessment

RDMM must establish the actual dependency condition.

Potential states include:

Active

Conditionally Active

Inactive but Configured

Dormant

Historical

Retained for Recovery

Transferred

Restricted

Eliminated

Unverified

Unknown

A dependency's observed inactivity does not establish that it cannot become active.

Active Residual Dependencies

An active residual dependency continues to participate in an operational

relationship.

Relevant elements include:

Observed Interaction

Execution Frequency

Dependent Function

Operational Environment

Affected Users or Systems

Failure Consequences

Responsible Function

Verification Evidence

Active dependencies require explicit disposition before retirement closure

can be evaluated.

Dormant Residual Dependencies

Dormant dependencies may remain configured or available without recent observed use.

Potential examples include:

Inactive API Credentials

Unused Scheduled Tasks

Legacy Recovery Scripts

Disabled Integrations

Standby Services

Emergency Interfaces

Dormancy must remain distinguishable from verified elimination.

Historical Dependencies

Historical dependencies may be necessary for:

Audit Reconstruction

Data Interpretation

Incident Investigation

Historical Reporting

Evidence Preservation

Legacy Recovery

Regulatory Record Access

Historical dependencies may be retained under explicit restrictions and responsibility rather than eliminated.

Dependency Significance

RDMM must assess the significance of each material residual dependency.

Relevant dimensions include:

Operational Criticality

Security Exposure

Data Importance

Recovery Importance

External Reliance

Physical-System Consequences

Authority Implications

Failure Impact

Resolution Complexity

Significance must reflect actual consequences rather than technical complexity alone.

Dependency Impact

RDMM must identify the consequences of dependency interruption, elimination, or continued existence.

Potential impacts include:

Service Failure

Data Inaccessibility

Security Exposure

Operational Disruption

Recovery Failure

External Consumer Failure

Loss of Historical Evidence

Physical Equipment Malfunction

Unauthorized Execution Capability

Impact assessment must support the selection of an appropriate dependency disposition.

Dependency Ownership

RDMM must identify the applicable ownership relationship.

Relevant elements include:

Dependency Object Owner

Retiring Software Owner

Dependent System Owner

Infrastructure Owner

Data Owner

External Provider

Shared Ownership Conditions

Ownership must not automatically be treated as proof of operational responsibility or authority.

Dependency Responsibility

Every material residual dependency must have an identifiable responsible function.

Relevant elements include:

Assessment Responsibility

Operational Responsibility

Resolution Responsibility

Security Responsibility

Data Responsibility

Monitoring Responsibility

Verification Responsibility

Escalation Responsibility

Responsibility must remain aligned with actual capability and applicable authority.

Dependency Authority

RDMM must identify authority for material dependency actions.

Relevant elements include:

Dependency Modification Authority

Access Revocation Authority

Interface Withdrawal Authority

Data Transfer Authority

Infrastructure Change Authority

Operational Restriction Authority

Dependency Transfer Authority

Resolution Approval Authority

External Approval Requirements

Authority to retire software does not automatically authorize termination of shared or externally owned dependencies.

Dependency Disposition

RDMM must establish the intended disposition of each material residual dependency.

Potential dispositions include:

Eliminate

Replace

Transfer

Restrict

Retain Under Governance

Monitor Pending Resolution

Defer Under Authorized Conditions

Escalate

Investigate Further

The selected disposition must remain justified and evidence-supported.

Dependency Elimination

Dependency elimination requires establishing that the relevant relationship has been terminated.

Relevant elements include:

Dependency Identity

Elimination Method

Affected Objects

Applicable Authority

Execution Evidence

Verification Method

Resulting State

Known Limitations

Removal of one technical connection must not automatically establish elimination of the complete dependency relationship.

Dependency Replacement

Dependency replacement occurs when a required relationship is satisfied through another object or mechanism.

Relevant elements include:

Original Dependency

Replacement Dependency

Required Capability

Compatibility Conditions

Operational Continuity

Verification Requirements

Residual Original Relationship

Replacement must be evaluated against the actual function of the original dependency.

Dependency Transfer

Dependency transfer changes the object, actor, or system responsible for

satisfying the dependency.

Relevant elements include:

Original Relationship

Successor Relationship

Transferred Function

Responsible Actors

Applicable Authority

Continuity Requirements

Transfer Evidence

Verification Result

Transfer does not establish resolution unless the intended relationship and responsibilities have been verified.

Dependency Restriction

Some dependencies may remain under controlled restrictions.

Potential restrictions include:

Read-Only Access

Network Isolation

Limited Execution

Restricted Credentials

Controlled Recovery Use

Limited User Access

Monitoring Requirements

Time-Limited Retention

Restrictions must be proportionate to the actual dependency and its risks.

Legitimate Dependency Retention

RDMM recognizes that some residual dependencies may legitimately continue.

Retention must preserve:

Retention Purpose

Dependency Identity

Operational Necessity

Responsible Function

Applicable Authority

Required Restrictions

Review Conditions

Termination Conditions

Supporting Evidence

Legitimate retention must not be confused with unresolved dependency neglect.

Residual Dependency Resolution

A dependency may be considered resolved when its intended disposition has been established and verified.

Resolution may include:

Verified Elimination

Verified Replacement

Verified Transfer

Authorized Restricted Retention

Authorized Historical Retention

Other Justified Governed Disposition

Resolution must remain associated with the actual dependency relationship and applicable evidence.

Dependency Resolution Preconditions

Material resolution actions may require:

Dependency Identity Established

Affected Systems Identified

Operational Consequences Assessed

Applicable Authority Established

Required Resources Available

Continuity Conditions Prepared

Recovery Requirements Addressed

Verification Method Defined

Unmet mandatory preconditions must remain explicit.

Dependency Resolution Verification

RDMM must verify whether the intended dependency disposition has been achieved.

Relevant elements include:

Intended Disposition

Actual Dependency State

Execution Evidence

Operational Observation

Dependent System Verification

Security Verification

Authority Verification

Remaining Obligations

Verification must distinguish observed absence from established termination where reactivation remains possible.

Dependency Elimination Evidence

Evidence of dependency elimination may include:

Configuration Removal

Credential Revocation

Interface Withdrawal

Execution Path Removal

Dependency Reference Removal

Runtime Verification

Dependent System Confirmation

Operational Monitoring

Access Verification

No single evidence type is universally sufficient.

The required evidence depends on the dependency's nature and operational

significance.

Residual Dependency Monitoring

Some dependencies require monitoring after disposition.

Potential monitoring concerns include:

Unexpected Reactivation

Unauthorized Access

Legacy Interface Calls

Scheduled Execution

Data Access Attempts

Dependency Failure

Security Events

Retention Expiration

Changed Operational Need

Monitoring must remain linked to an identifiable responsible function.

Dependency Reactivation

A previously inactive or restricted dependency may become active again.

RDMM must preserve:

Reactivation Event

Dependency Identity

Observed Activity

Originating Actor or System

Applicable Authority

Operational Consequences

Required Response

Reassessment Evidence

Reactivation may invalidate an earlier dependency disposition or retirement assumption.

Residual Dependency Risk

RDMM must identify risks associated with unresolved or retained dependencies.

Potential risks include:

Security Exposure

Operational Failure

Uncontrolled Access

Data Loss

Dependency Reactivation

Recovery Failure

External Consumer Disruption

Physical-System Consequences

Unassigned Responsibility

Risk classification must remain evidence-supported and distinguishable from risk acceptance.

Residual Risk Acceptance

Where residual risk is accepted under applicable governance, RDMM must preserve:

Risk Identity

Affected Dependency

Risk Basis

Applicable Authority

Acceptance Conditions

Operational Restrictions

Responsible Function

Review Requirements

Expiration or Reassessment Conditions

Acceptance must not be inferred from inaction or lack of observed incidents.

Dependency Exception Governance

An exception may be required where ordinary dependency resolution conditions cannot be satisfied.

Relevant elements include:

Exception ID

Dependency Identity

Reason

Applicable Authority

Alternative Control

Residual Risk

Validity Conditions

Review Requirements

Evidence

An exception must not be used to conceal an unresolved dependency.

Dependency Uncertainty

RDMM must preserve uncertainty where dependency conditions cannot be established.

Potential conditions include:

Unknown Consumer

Unverified Execution Path

Incomplete Dependency Mapping

Uncertain Data Relationship

Unknown Credential Use

Unverified Recovery Dependency

Conflicting Operational Evidence

Uncertain Resolution Outcome

Material uncertainty must remain visible in retirement closure assessments.

Dependency Reassessment

RDMM must support reassessment when:

New Dependency Evidence Appears

Operational Conditions Change

Migration Outcomes Change

Support Conditions Change

Previously Dormant Dependencies Reactivate

Security Conditions Change

Responsible Functions Change

Retention Conditions Expire

Resolution Verification Fails

Reassessment must preserve historical dependency states and decisions.

Residual Dependency Classification

RDMM recognizes the following governance classifications:

Residual Dependency Identified

Assessment Pending

Active

Conditionally Active

Dormant

Historical

Retained for Recovery

Resolution Planned

Resolution Authorized

Resolution Active

Transferred

Replaced

Restricted

Retained Under Governance

Eliminated — Verification Pending

Verified Eliminated

Verified Replaced

Verified Transferred

Governed Retention Established

Unresolved

Exception Active

Reassessment Required

Closed

Unknown

Conflicted

Each classification must remain connected to the dependency's scope, evidence, responsible function, and applicable time.

Residual Dependency Closure

A residual dependency case may be closed when its disposition has been established through appropriate evidence.

Closure must preserve:

Dependency Case Identity

Retiring Software Reference

Dependency Relationship

Intended Disposition

Actual Disposition

Verification Evidence

Responsible Function

Residual Obligations

Applicable Authority

Closure Decision

A dependency case may be closed with governed retention where the continuing relationship is legitimate and appropriately controlled.

Dependency closure must not independently establish complete software retirement.

Governance Requirements

A conforming RDMM implementation must satisfy the following requirements. Dependency Identity and Discovery

• Establish a stable residual dependency case identity

• Identify the retiring software object

• Identify the dependent or supporting object

• Establish dependency direction

• Identify dependency purpose

• Classify dependency type

• Preserve dependency origin where available

• Establish an appropriate discovery process

• Preserve material discovery evidence

• Identify discovery uncertainty

Dependency State and Significance

• Establish the current dependency state

• Distinguish active, dormant, historical, and eliminated conditions

• Identify direct dependencies

• Identify indirect and transitive dependencies

• Evaluate operational significance

• Identify dependency impact

• Identify continuing obligations

• Preserve relevant operational evidence

Responsibility and Authority

• Identify dependency ownership

• Assign operational responsibility

• Identify resolution responsibility

• Establish applicable authority

• Identify shared or external dependency constraints

• Preserve authority boundaries

• Identify escalation requirements

• Prevent software retirement authority from being treated as unrestricted dependency modification authority.

Dependency Disposition

• Establish the intended dependency disposition

• Identify resolution requirements

• Evaluate elimination feasibility

• Evaluate replacement or transfer where applicable

• Establish retention justification where applicable

• Establish required restrictions

• Identify mandatory resolution preconditions

• Preserve disposition decisions

• Identify residual risk

• Preserve applicable exceptions

Verification and Continuity

• Verify material dependency dispositions

• Preserve resolution evidence

• Identify unresolved relationships

• Establish monitoring where required

• Identify reactivation conditions

• Preserve reassessment triggers

• Classify resulting dependency status

• Establish dependency closure eligibility

• Preserve continuing responsibility for retained dependencies

• Transfer relevant evidence to retirement closure governance

• Prevent dependency inactivity from substituting for verified

• Prevent dependency closure from substituting for complete software

elimination. retirement closure.

Governance Logic

Retirement Context → Residual Dependency Discovery → Dependency Case

Identity → Source Software Identification → Dependent Object Identification → Dependency Origin → Dependency Direction → Dependency Purpose → Dependency Classification → Actual Dependency State → Operational Significance → Impact Assessment → Responsibility → Applicable Authority → Intended Disposition → Resolution Preconditions → Governed Resolution / Transfer / Restriction / Retention → Verification → Residual Risk → Resulting Dependency State → Dependency Closure → Software Residual Dependency Profile

Where elimination is required:

Residual Dependency → Elimination Requirements → Authorized Action →

Termination Evidence → Verification → Verified Eliminated

Where replacement is required:

Residual Dependency → Replacement Object → Compatibility Assessment →

Governed Transition → Verification → Verified Replaced

Where transfer is required:

Residual Dependency → Successor Relationship → Authority and

Responsibility Alignment → Transfer → Continuity Verification → Verified Transferred

Where retention is legitimate:

Residual Dependency → Retention Purpose → Authority → Restrictions →

Responsible Function → Review Conditions → Governed Retention

Where the dependency is uncertain:

Potential Dependency → Evidence Gap → Further Discovery → Updated

Classification → Disposition Assessment

Where a dependency reactivates:

Previously Inactive Dependency → Reactivation Evidence → Impact

Reassessment → Revised Disposition → Updated Governance State

RDMM must not equate software withdrawal with dependency elimination.

Governance Outputs

The principal output is:

Software Residual Dependency Profile

Supporting outputs may include:

Residual Dependency Case Identity

Residual Dependency Register

Dependency Relationship Map

Dependency Origin Record

Dependency Direction Profile

Dependency State Assessment

Dependency Significance Assessment

Dependency Impact Record

Dependency Ownership Record

Dependency Responsibility Profile

Dependency Authority Record

Dependency Disposition Plan

Dependency Resolution Requirements

Dependency Replacement Record

Dependency Transfer Record

Dependency Restriction Record

Governed Retention Profile

Dependency Verification Record

Residual Risk Register

Dependency Exception Register

Dependency Monitoring Record

Dependency Reactivation Record

Dependency Closure Record

Minimum Implementation Framework

A minimum RDMM implementation must establish eight governance layers. Layer 1 — Dependency Identity

Identify the retiring software, dependent objects, dependency direction,

purpose, and case reference. Layer 2 — Dependency Discovery

Establish a proportionate discovery process covering material technical,

operational, data, interface, security, authority, and recovery relationships. Layer 3 — State and Significance

Determine the actual dependency state, operational significance, potential

consequences, and known uncertainty. Layer 4 — Responsibility and Authority

Identify ownership, responsible functions, required capabilities, and

applicable authority for dependency actions. Layer 5 — Disposition Selection

Establish whether each dependency must be eliminated, replaced, transferred, restricted, retained, or further investigated. Layer 6 — Governed Resolution

Execute or coordinate authorized dependency disposition activities under

appropriate preconditions. Layer 7 — Verification and Monitoring

Verify the resulting dependency condition, preserve evidence, identify

residual risk, and monitor retained relationships where required. Layer 8 — Closure and Continuity

Maintain the Software Residual Dependency Profile, preserve continuing

obligations, and provide evidence to retirement closure governance.

The framework may be implemented through dependency inventories, configuration management systems, runtime monitoring, security tools, infrastructure platforms, manual governance processes, or AI-assisted analysis.

The implementation mechanism must adapt to the actual dependency relationships.

Cross-Module Boundary

RPM → RDMM

RPM identifies dependencies relevant to retirement planning and establishes

initial resolution requirements.

RDMM governs the actual residual dependency conditions and their

dispositions.

ESMM → RDMM

ESMM identifies support-related dependencies and continuing operational

conditions after support changes.

RDMM governs residual dependency relationships that remain relevant to retirement.

SMM → RDMM

SMM identifies dependencies that remain after software migration, including incomplete transfers and continuing source reliance.

RDMM governs their classification, resolution, transfer, restriction, or

retention.

RDMM → RCCLM

RDMM provides evidence of residual dependency disposition and continuing

obligations.

RCCLM determines whether the resulting dependency conditions are compatible with legitimate retirement closure.

Residual dependency governance establishes the disposition of continuing

relationships. It does not independently establish retirement finality.

Cross-Architecture Boundary

RDMM governs residual dependency relationships within SRTS.

It does not replace general software dependency governance or other specialized lifecycle mechanisms.

SRS — Software Requirements Standard

Governs requirements associated with replacement or continuing software

capabilities.

SCS — Software Creation Standard

Governs creation of software required to replace or resolve dependencies.

SDS — Software Dependency Standard

Governs general dependency identity, origin, suitability, change,

continuity, and risk.

SCFS — Software Configuration Standard

Governs configuration and artifact relationships relevant to dependency

changes.

SQS — Software Quality Standard

Governs quality requirements and verification of modified or replacement

software.

SSS — Software Security Standard

Governs security exposure, vulnerabilities, and remediation.

SRLS — Software Release Standard

Governs release conditions for replacement components.

SDPS — Software Deployment Standard

Governs deployment, production state, and rollback or recovery.

SCMS — Software Change Management Standard

Governs changes required to resolve or transfer dependencies.

SLCS — Software Lifecycle Control Standard

Governs lifecycle state consistency, transition control, exceptions, and

continuity.

RDMM may also interact with applicable OOF® authority, accountability, identity, operational reality, data, autonomous systems, and physical-system governance mechanisms.

It does not replace their specialized governance functions.

Practical Applications

Enterprise Application Retirement

Identifying historical reporting, data access, and integration dependencies

that remain after application withdrawal.

Cloud Service Decommissioning

Managing residual API consumers, shared infrastructure, credentials, and

external service relationships.

Software Supply-Chain Modernization

Identifying remaining references to obsolete libraries, runtimes, and

unsupported components.

AI System Retirement

Governing residual model dependencies, inference endpoints, orchestration

workflows, and persistent access relationships.

Autonomous Agent Decommissioning

Managing scheduled tasks, tool permissions, memory references, and

delegated authority that survive agent execution termination.

Industrial Software Retirement

Identifying dependencies between retiring software, firmware, controllers,

and physical equipment.

Critical Infrastructure

Preserving recovery capabilities, shared dependencies, and continuing

operational responsibilities during software withdrawal.

Practical Use Case 1 — Residual Dependencies After Cloud Service

Retirement

An organization migrates a business application from a legacy cloud service

to a new platform.

The replacement application is operational, and the original cloud service is scheduled for deletion.

A dependency review identifies that several external systems still reference the legacy service.

RDMM evaluates:

Retiring Service → External Consumers → Interface Dependencies → Actual

Usage → Operational Impact → Required Disposition

The assessment identifies:

• One external partner continues sending requests to the legacy API

• A scheduled reporting process still retrieves historical data from the

• An unused service account retains privileged access

• A backup process depends on a legacy storage configuration

• Some consumers have already migrated, but their old connection

original environment. settings remain configured.

RDMM establishes separate dispositions:

External API Consumer → Transfer to Replacement Interface

Historical Reporting Dependency → Verified Alternative Data Access

Privileged Service Account → Authorized Revocation

Backup Dependency → Recovery Continuity Assessment

Dormant Connection Settings → Verified Removal or Governed Restriction

The retirement case cannot treat all these relationships as eliminated

merely because the primary application has migrated.

Primary Service Replaced ≠ Residual Dependencies Resolved.

Practical Use Case 2 — Residual Authority After Autonomous Agent

Retirement

An organization retires an autonomous software maintenance agent.

The agent's primary execution environment has been deactivated, and a

successor agent now performs the intended maintenance functions.

A subsequent assessment identifies continuing relationships associated with

the original agent.

RDMM evaluates:

Retired Agent Identity → Tool Permissions → Scheduled Actions → External

Credentials → Persistent References → Actual Dependency State

The assessment identifies:

• Repository credentials remain valid

• A scheduled automation task can still invoke an external maintenance

• An integration retains the original agent identity as an authorized

• Historical task records remain necessary for audit reconstruction

• The successor agent has its own separately authorized permissions

workflow. actor.

RDMM establishes the required dispositions:

Repository Credentials → Verified Revocation

Scheduled Automation → Verified Termination or Transfer

External Authorization → Authority Relationship Reconciliation

Historical Task Records → Governed Retention

Successor Permissions → Independent Authority Verification

The original agent's retirement cannot be considered fully reconciled while

its authority relationships remain active without a legitimate continuing purpose.

Agent Execution Terminated ≠ Agent Authority Eliminated.

Critical Distinctions

Software Removed ≠ Dependency Eliminated.

Software Retired Administratively ≠ Dependencies Resolved Operationally.

Dependency Identified ≠ Dependency Verified.

Dependency Listed ≠ Dependency Understood.

Direct Dependency Identified ≠ Indirect Dependencies Identified.

No Observed Activity ≠ No Remaining Dependency.

Dependency Dormant ≠ Dependency Eliminated.

Connection Disabled ≠ Relationship Terminated.

Artifact Removed ≠ Historical Obligation Eliminated.

Dependency Transferred ≠ Transfer Verified.

Dependency Replaced ≠ Replacement Suitable.

Dependency Restricted ≠ Dependency Eliminated.

Dependency Retained ≠ Dependency Ungoverned.

Dependency Ownership ≠ Resolution Authority.

Responsibility Assigned ≠ Resolution Capability Established.

Technical Access ≠ Legitimate Authority.

Agent Execution Stopped ≠ Agent Permissions Revoked.

Credential Unused ≠ Credential Invalid.

Resolution Executed ≠ Resolution Verified.

Dependency Case Closed ≠ Software Retirement Closed.

Canonical Boundary

Residual Dependency Management Module (RDMM) governs the identification,

classification, responsibility, authority, disposition, verification, and continuing control of dependencies that remain relevant during or after software retirement activities.

It establishes the governance relationship between the retiring software

object and the technical, operational, data, interface, infrastructure, security, identity, authority, recovery, and physical-system relationships that may survive its withdrawal.

RDMM does not assume that every residual dependency must be eliminated.

It permits justified retention, restriction, replacement, or transfer where the resulting relationship remains identifiable, appropriately authorized, and governed.

RDMM does not independently authorize software retirement, perform migration governance, or determine final retirement closure.

Its principal governance output is the:

Software Residual Dependency Profile

This output establishes what remains connected to the retiring software,

why the relationship matters, what disposition is required, what has been verified, and which obligations continue.

Canonical Closing Statement

Software can disappear from an operational environment while its dependencies remain embedded in the systems around it.

A deleted application may leave behind active integrations.

A retired service may continue receiving external requests.

A migrated database may retain historical relationships that depend on legacy software.

A discontinued component may remain referenced by recovery systems.

An industrial device may still require the firmware associated with a retired platform.

And an autonomous agent may stop executing while its credentials, scheduled tasks, and delegated authority remain active.

These conditions reveal a fundamental distinction between the state of

software and the state of the relationships surrounding it.

RDMM establishes the governance structure required to preserve that distinction.

It makes residual dependencies identifiable even when the original software is no longer visibly active.

It evaluates their actual operational significance and continuing

consequences.

It assigns responsibility and preserves authority boundaries.

It distinguishes elimination from replacement, transfer, restriction, and

legitimate retention.

It requires material dispositions to be verified rather than assumed.

And it preserves unresolved relationships as explicit governance conditions

instead of allowing them to disappear behind an administrative retirement status.

Retirement does not require every historical relationship to disappear. It requires every material continuing relationship to have a legitimate and verifiable governance disposition.

Residual Dependency Management Module (RDMM) transforms the hidden

remainder of software retirement into an identifiable, accountable, and evidence-supported governance space—ensuring that what survives the software remains governed.